

Python for Computer Science and Data Science 2 (CSE 3652)

MAJOR ASSIGNMENT-2: RECURSION, SEARCHING, SORTING AND BIG O

1. Write a recursive function to compute the sum of the first n natural numbers. Then, analyze its time complexity using Big O notation.

Tasks:

- (a) Implement a recursive function `recursive_sum(n)` that computes the sum of the first n natural numbers using recursion.
- (b) Determine the base case and recursive step.
- (c) Write a function `iterative_sum(n)` that computes the sum using a loop.
- (d) Compare the time complexity of both functions using Big O notation.
- (e) Explain why recursion is less efficient in this case compared to iteration.

Deliverables:

- (a) Python code for `recursive_sum(n)` and `iterative_sum(n)`.
- (b) A written explanation of the time complexity of both functions.
- (c) A comparison between recursion and iteration in this problem.

```
In [1]: ## 1(a)
def recursive_sum(n):
    if n == 0:
        return 0
    return n + recursive_sum(n - 1)

n = int(input("Enter n: "))
print("Recursive Sum:", recursive_sum(n))
```

Recursive Sum: 55

```
In [2]: ## 1(b)
def recursive_sum(n):
    if n == 0:
        return 0
    return n + recursive_sum(n - 1)

print("Base case: n == 0 returns 0")
print("Recursive step: n + recursive_sum(n-1)")
```

Base case: $n == 0$ returns 0
Recursive step: $n + \text{recursive_sum}(n-1)$

```
In [3]: ## 1(c)
def iterative_sum(n):
    total = 0
    for i in range(1, n + 1):
        total += i
    return total

n = int(input("Enter n: "))
print("Iterative Sum:", iterative_sum(n))
```

Iterative Sum: 55

```
In [4]: ## 1(d)
print("Recursive Time Complexity:  $O(n)$ ")
print("Iterative Time Complexity:  $O(n)$ ")
```

Recursive Time Complexity:
 $O(n)$ Iterative Time
Complexity: $O(n)$

```
In [5]: ## 1(e)
```

```
print("Iteration is more efficient due to no function call overhead")
```

Iteration is more efficient due to no function call overhead

2. The **Josephus Problem** is a famous theoretical problem related to recursion and combinatorics. It is defined as follows:

- There are n people standing in a circle, numbered from 0 to $n - 1$.
- Every k -th person is eliminated in a circular manner until only one person remains.
- Your task is to implement a recursive function to find the position of the last remaining person (zero-based index).

Tasks:

- (a) Implement a recursive function `josephus(n, k)` that returns the position of the last person remaining.
- (b) Identify the base case and the recursive formula.
- (c) Analyze the time complexity of your recursive implementation using Big O notation.
- (d) Implement an iterative version and compare its efficiency with the recursive version.
- (e) Test the function with different values of n and k .

Deliverables:

- Python code for `josephus(n, k)` using recursion.
- Explanation of the recursive logic and base case.
- Big O complexity analysis.
- Comparison of recursion vs. iteration for solving the problem.

```
In [6]: ## 2(a)
def josephus(n, k):
    if n == 1:
        return 0
    return (josephus(n - 1, k) + k) % n

n = int(input("Enter n: "))
k = int(input("Enter k: "))
print("Last remaining position:", josephus(n, k))
```

Last remaining position: 2

```
In [7]: ## 2(b)
print("Base case: n == 1 returns 0")
print("Recursive formula: (J(n-1, k) + k) % n")
```

Base case: $n == 1$ returns 0
Recursive formula: $(J(n-1, k) + k) \% n$

```
In [8]: ## 2(c)
print("Time Complexity: O(n)")
```

Time Complexity: $O(n)$

```
In [9]: ## 2(d)
def josephus_iterative(n, k):
    result = 0
    for i in range(2, n + 1):
        result = (result + k) % i
    return result

n = int(input("Enter n: "))
k = int(input("Enter k: "))
print("Iterative Result:", josephus_iterative(n, k))
```

Iterative Result: 2

```
In [10]: ## 2(e)
def josephus(n, k):
    if n == 1:
        return 0
    return (josephus(n - 1, k) + k) % n

def josephus_iterative(n,
k): result = 0
for i in range(2, n + 1):
    result = (result + k) % i
return result

tests = [(5,2), (7,3), (10,2)]

for n, k in tests:
    print("n =", n, "k =", k)
    print("Recursive:", josephus(n, k))
    print("Iterative:", josephus_iterative(n, k))
    print()
```

```
n = 5 k = 2
Recursive: 2
Iterative: 2
```

```
n = 7 k = 3
Recursive: 3
Iterative: 3
```

```
n = 10 k = 2
Recursive: 4
Iterative: 4
```

3. You need to implement an **efficient recursive function** to compute a^b (where a is the base and b is the exponent) using **Exponentiation by Squaring**, which reduces the number of recursive calls significantly.

Tasks:

- Implement a recursive function `fast_power(a, b)` that computes a^b using **divide and conquer**.
- Identify the base case and the recursive formula:
 - If b is even, use the identity:
$$a^b = (a^{b/2})^2$$
 - If b is odd, use the identity:
$$a^b = a \times a^{b-1}$$
- Compare the time complexity of the standard recursive method ($O(b)$) and the optimized exponentiation by squaring method ($O(\log b)$).
- Implement an iterative version of exponentiation by squaring and compare it with the recursive implementation.
- Test your function with large values of b (e.g., $a = 2, b = 1000000$) and observe the performance.

Deliverables:

- Python code for `fast_power(a, b)` using recursion.
- Explanation of divide-and-conquer logic used in exponentiation by squaring.
- Big O complexity analysis for both naive recursion and optimized recursion.
- Comparison with an iterative version of the algorithm.

```
In [11]: ## 3(a)
def fast_power(a, b):
    if b == 0:
```

```
return 1
if b % 2 == 0:
    half = fast_power(a, b // 2)
    return half * half
return a * fast_power(a, b - 1)

a = int(input("Enter a: "))
b = int(input("Enter b: "))
print("Result:", fast_power(a, b))
```

Result: 1024

```
In [12]: ## 3(b)
print("Base case: b == 0 returns 1")
print("If b even: (a^(b/2))^2")
print("If b odd: a * a^(b-1)")
```

Base case: b == 0 returns
1 If b even: (a^(b/2))^2
If b odd: a * a^(b-1)

```
In [13]: ## 3(c)
print("Naive Recursion: O(b)")
print("Exponentiation by Squaring: O(log b)")
```

Naive Recursion: O(b)
Exponentiation by Squaring: O(log b)

```
In [14]: ## 23(d)
def fast_power_iterative(a,
    b): result = 1
    while b > 0:
        if b % 2 ==
            1: result
                *= a
            a *= a
            b //= 2
    return result

a = int(input("Enter a: "))
b = int(input("Enter b: "))
print("Iterative Result:", fast_power_iterative(a, b))
```

Iterative Result: 1024

```
In [15]: ## 3(e)
def fast_power_iterative(a,
    b): result = 1
    while b > 0:
        if b % 2 ==
            1: result
                *= a
            a *= a
            b //= 2
    return result

import math

print("Testing a = 2, b = 1000000")

digits = int(b * math.log10(a)) + 1
print("Number of digits:", digits)

last_digits = fast_power_iterative(2, 1000000) % (10**10)
print("Last 10 digits:", last_digits)
```

Testing a = 2, b = 1000000
Number of digits: 4
Last 10 digits: 2747109376

Phantom tracking

A notorious hacker known as **The Phantom** has been manipulating digital transactions in Bhubaneswar to move stolen money through multiple accounts.

The Cyber Crime Unit has intercepted a list of transactions (in thousands of rupees) from different accounts.

However, the transactions are unordered, some are duplicates, and The Phantom has hidden key details among normal transactions.

Your task is to analyze the data step by step, finally leading to the criminal.

Objective 1) Identifying Suspicious Transactions

The police believe that certain high-value transactions (above a given threshold) are linked to the criminal network.

Task: Use **Linear Search** to extract all suspicious transactions above a certain amount.

Given Data:

transactions = [120, 45, 300, 220, 90, 600, 130, 75, 800, 500, 350, 40]
threshold = 250

Your goal is to identify transactions greater than ₹250,000 to narrow the suspect list.

Once you extract the high-value transactions, you will use them for the next step.



```
In [16]: transactions = [120, 45, 300, 220, 90, 600, 130, 75, 800, 500, 350, 40]
threshold = int(input("Enter threshold: "))

def linear_search(transactions,
                  threshold): result = []
    for t in transactions:
        if t > threshold:
            result.append(t)
    return result

print("Suspicious Transactions:", linear_search(transactions, threshold))
```

Suspicious Transactions: [300, 600, 800, 500, 350]

Objective 2) Organizing the Transaction Data

The extracted suspicious transactions from Objective 1 need to be sorted in ascending order for further analysis.

Task:

Use **Selection Sort** to sort the suspicious transactions and return a sorted list of transactions.

Once sorted, the police will use these transactions to identify frequent recipients.



```
In [17]: def selection_sort(arr): n = len(arr)
    for i in range(n): min_idx = i
    for j in range(i + 1, n):
        if arr[j] < arr[min_idx]: min_idx = j
```

```
arr[i], arr[min_idx] = arr[min_idx], arr[i]
return arr

data = [300, 600, 800, 500, 350]
print("Sorted Transactions:", selection_sort(data))
```

Sorted Transactions: [300, 350, 500, 600, 800]

Objective 3) Finding a Specific Transaction

A forensic accountant identified a key transaction—one of these amounts was transferred to an account linked to The Phantom.

Task:

Use **Binary Search** to check if a specific suspicious transaction exists in the sorted list.

search_amount = 500

Return True if the ₹500,000 transaction is in the suspicious list, otherwise return False.

If this transaction exists, it will lead to a secret offshore account linked to The Phantom.



```
In [18]: def binary_search(arr,
        target): left = 0
        right = len(arr) - 1
        while left <= right:
            mid = (left + right) // 2
            if arr[mid] == target:
                return True
            elif arr[mid] <
                target: left =
                mid + 1
            else:
                right = mid - 1
        return False

arr = [300, 350, 500, 600, 800]
target = int(input("Enter value to search: "))
print("Found:", binary_search(arr, target))
```

Found: True

Objective 4) Reconstructing the Criminal's Full Transaction History

The police have recovered all transaction records from The Phantom's secret bank. However, they are completely unorganized and need to be sorted for further analysis.

Task:

Use Merge Sort to sort all transactions.

Input: transactions = [120, 45, 300, 220, 90, 600, 130, 75, 800, 500, 350, 40]

Your goal is to return a fully sorted list so the police can analyze the transaction history and track The Phantom.

Once sorted, the police notice a repeating pattern of money being sent to a specific name— The Phantom's real identity!

```
In [19]: def merge_sort(arr):
        if len(arr) <= 1:
            return arr
        mid = len(arr) // 2
        left = merge_sort(arr[:mid])
        right = merge_sort(arr[mid:])
        return merge(left, right)
```

```
def merge(left, right):  
    result = []  
    i = 0  
    j = 0  
    while i < len(left) and j < len(right):  
        if left[i] < right[j]:  
            result.append(left[i]) i += 1  
        else:  
            result.append(right[j]) j += 1  
    result.extend(left[i:]) result.extend(right[j:])  
    return result  
  
transactions = [120, 45, 300, 220, 90, 600, 130, 75, 800, 500, 350, 40]  
print("Fully Sorted Transactions:", merge_sort(transactions))
```

Fully Sorted Transactions: [40, 45, 75, 90, 120, 130, 220, 300, 350, 500, 600, 800]

Submitted By-

- Name- Amulya Shrivastava
- Regd No- 2341019456
- Section- 23412F3
- Serial No- 59
- Subject- PFCD2 Major Assignment-02